



UNIVERSITÀ DEGLI STUDI DI PALERMO

DIPARTIMENTO DI MATEMATICA E INFORMATICA

CORSO DI LAUREA IN INFORMATICA

Applicazioni della teoria delle categorie all'informatica teorica

COINDUZIONE E COALGEBRE

TESI DI LAUREA DI
SAMUELE MARIA MORREALE

RELATORE
PROF.SSA GIUSEPPA CASTIGLIONE

ANNO ACCADEMICO 2025 - 2026

TRIENNALE



Indice

1	Introduzione	2
2	Teoria delle categorie	4
3	Coalgebre	9
3.1	Coalgebra finale e coricorsione	9
3.2	Coinduzione come principio di dimostrazione	11
3.3	Bisimulazione	11
3.4	Predicati e invarianti	12
3.5	Operatori temporali	14
4	Esempi pratici di coalgebre in informatica teorica	15
4.1	Linguaggi regolari	15
4.1.1	Il funtore degli automi	15
4.1.2	Automi come riconoscitori di linguaggi	16
4.1.3	La coalgebra finale dei linguaggi	16
4.1.4	Chiusura dei linguaggi regolari	17
4.1.5	Chiusura per concatenazione e stella di Kleene	19
4.2	Grammatiche libere dal contesto	23
4.2.1	Il funtore degli alberi di derivazione	23
4.2.2	Grammatiche come coalgebre	24
4.2.3	Proprietà strutturali tramite operatori temporali	24
4.2.4	Ricorsività	26
4.3	Macchine di Turing	27
4.3.1	Il funtore delle macchine di Turing	27
4.3.2	Macchine di Turing come coalgebre	28
4.3.3	Accettazione non deterministica tramite $\diamond$	28
4.3.4	Bisimulazione ed equivalenza di macchine	29

1. Introduzione

L'informatica teorica si occupa di due tipi di oggetti profondamente diversi tra loro. Da un lato i **dati**: strutture finite, costruite a partire da componenti più semplici in un numero finito di passi. Dall'altro ci sono i **processi**: sistemi che evolvono nel tempo, che possono non terminare mai, e la cui natura sta in come si comportano quando vengono osservati.

La matematica classica offre uno strumento potente per ragionare sul primo tipo di oggetti: l'**induzione**. La struttura induttiva permette sia di *definire* funzioni per ricorsione sulla struttura dei dati, sia di *dimostrare* proprietà per induzione.

I processi, però, non si lasciano descrivere altrettanto bene da questo stesso strumento: questi oggetti non si costruiscono a partire da casi base, perché non hanno un caso base. Ciò che li caratterizza è *cosa succede quando li si osserva*: quale simbolo producono, in quale stato transitano, se si arrestano o continuano. Per ragionare su oggetti di questo tipo serve un principio duale all'induzione, la **coinduzione**, che permetta di definire funzioni per *coricorsione*, specificandone il comportamento passo dopo passo anziché costruendole dal basso, e di dimostrare l'uguaglianza di due comportamenti mostrando che sono osservativamente indistinguibili, invece di scomporli nei loro costituenti.

La **teoria delle categorie** fornisce il linguaggio in cui questa dualità tra induzione e coinduzione si può esprimere in modo del tutto preciso e generale. Un tipo di dato induttivo è catturato dalla nozione di **algebra iniziale** per un funtore: la struttura più semplice che si può costruire a partire da un insieme di costruttori. Un sistema di transizione, dualmente, è catturato dalla nozione di **coalgebra**, e il suo comportamento osservabile è descritto dalla **coalgebra finale**: la struttura più informativa verso cui ogni altro sistema dello stesso tipo si proietta in modo unico.

Questa prova finale si propone di sviluppare, in modo autocontenuto, gli strumenti categorici necessari a formalizzare questo punto di vista, e di applicarli a tre esempi classici dell'informatica teorica. Il capitolo 2 introduce i concetti di base della teoria delle categorie — categorie, funtori, isomorfismi, oggetti finali — necessari a dare un significato preciso alla nozione di coalgebra. Il capitolo 3 sviluppa la teoria generale delle coalgebre: la definizione di coalgebra e dei suoi omomorfismi, la coalgebra finale come oggetto universale, i principi di coricorsione e coinduzione che essa induce e la nozione di bisimulazione come criterio di equivalenza comportamentale. Il capitolo 4, infine, applica questi strumenti a tre ambiti concreti dell'informatica teorica — i linguaggi regolari, le grammatiche libere dal contesto e le macchine di Turing — mostrando come proprietà classiche di ciascuno di questi formalismi

(chiusura dei linguaggi regolari, raggiungibilità e produttività dei simboli di una grammatica, correttezza di simulazioni tra varianti di macchine di Turing) si lascino esprimere e dimostrare naturalmente nel linguaggio delle coalgebre, degli operatori temporali coinduttivi e della bisimulazione.

2. Teoria delle categorie

Tutto lo studio delle coalgebre nasce dai concetti fondamentali della teoria delle categorie [4], il cui oggetto fondamentale è dato dalle categorie.

Definizione 2.1 (Categoria). Una **categoria** C consiste dei seguenti dati:

- una collezione di oggetti $\text{Ob}(C)$. Si usa la notazione $X \in C$ per indicare $X \in \text{Ob}(C)$.
- una collezione di morfismi $\text{Mor}(C)$. Ad ogni morfismo di C , che viene indicato come $f : X \rightarrow Y$ o come $X \xrightarrow{f} Y$, sono associati un oggetto dominio $X \in C$ e un oggetto codominio $Y \in C$. La collezione di morfismi che hanno dominio X e codominio Y si indica con $\text{Hom}_C(X, Y)$ oppure, se la categoria di riferimento è implicita, con $\text{Hom}(X, Y)$.

Per ogni coppia di morfismi $X \xrightarrow{f} Y$ e $Y \xrightarrow{g} Z$ è definita un'operazione di *composizione di morfismi*, indicata come $g \circ f : X \rightarrow Z$ o semplicemente $gf : X \rightarrow Z$, che soddisfa due condizioni:

1. Associatività: Dati tre morfismi $W \xrightarrow{f} X$, $X \xrightarrow{g} Y$, $Y \xrightarrow{h} Z$, la loro composizione è associativa, cioè $h \circ (g \circ f) = (h \circ g) \circ f$, e può essere indicata più semplicemente come $h \circ g \circ f$ o hgf .
2. Morfismo identità: Per ogni oggetto $X \in C$, esiste un *morfismo identità*, indicato con $\text{id}_X : X \rightarrow X$, che è un elemento neutro per la composizione: dato $f : X \rightarrow Y$, si ha che $f \circ \text{id}_X = f = \text{id}_Y \circ f$.

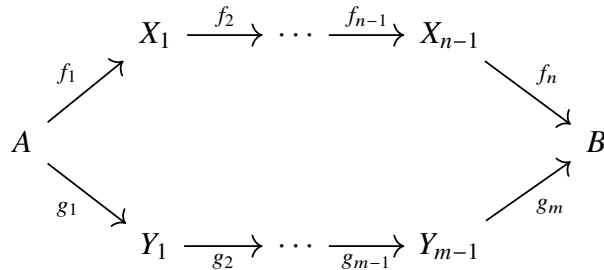
Una categoria si dice *piccola* se $\text{Ob}(C)$ è un insieme, altrimenti si dice *grande*. Se $\text{Hom}_C(X, Y)$ è un insieme per qualunque coppia di oggetti X, Y , allora la categoria si dice *localmente piccola*.

L'esempio più immediato di categoria, e l'unica categoria esplorata più avanti, è la categoria degli insiemi.

Esempio 2.1. La categoria degli insiemi **Set** ha come oggetti gli insiemi e le funzioni tra gli insiemi come morfismi. Dato che la collezione degli insiemi non è essa stessa un insieme, questa categoria non è piccola, ma, essendo $\text{Hom}_{\text{Set}}(X, Y)$ l'insieme delle funzioni da X a Y , è localmente piccola. La composizione di morfismi è data dalla composizione di funzioni $g \circ f$ e il morfismo identità id_X è la funzione identità sull'insieme X .

Le proprietà categoriche sono espresse in termini di morfismi, spesso rappresentati tramite diagrammi. Due aspetti fondamentali in questo senso sono la commutatività e l'unicità:

Commutatività: Equazioni del tipo $f_n \circ f_{n-1} \circ \dots \circ f_2 \circ f_1 = g_m \circ g_{m-1} \circ \dots \circ g_2 \circ g_1$ sono rappresentate da diagrammi come il seguente.



Unicità: Una formulazione comune in teoria delle categorie è: per ogni elemento esiste un unico morfismo $f : X \rightarrow Y$ che soddisfa una certa proprietà. Un morfismo di questo tipo è indicato nei diagrammi come $f : X \dashrightarrow Y$.

Un concetto derivato immediatamente dalla struttura categorica è quello di isomorfismo.

Definizione 2.2 (Isomorfismo). Un morfismo $f : X \rightarrow Y$ in una categoria $\mathcal{C}$ è un **isomorfismo** se esiste un morfismo $g : Y \rightarrow X$ tale che $g \circ f = \text{id}_X$ e $f \circ g = \text{id}_Y$. In tal caso g è unico ed è detto *inverso* di f , scritto f^{-1} , e si dice che X e Y sono *isomorfi*, scritto $X \cong Y$.

Nella categoria **Set** gli isomorfismi sono esattamente le funzioni biunivoche. La nozione di isomorfismo è fondamentale in tutta la teoria delle categorie e nel contesto delle coalgebre: la coalgebra finale è unica a meno di isomorfismo, e la sua struttura di transizione è essa stessa un isomorfismo [4].

Un altro oggetto della teoria delle categorie fondamentale per lo studio delle coalgebre è dato dai funtori.

Definizione 2.3 (Funtore). Date due categorie $\mathcal{C}$ e $\mathcal{D}$, un **funtore** $F : \mathcal{C} \rightarrow \mathcal{D}$ consiste di due corrispondenze $\text{Ob}(\mathcal{C}) \rightarrow \text{Ob}(\mathcal{D})$ e $\text{Mor}(\mathcal{C}) \rightarrow \text{Mor}(\mathcal{D})$, indicate entrambe con F , tali che:

1. F rispetta domini e codomini: data $f : X \rightarrow Y$ in $\mathcal{C}$, allora $F(f) : F(X) \rightarrow F(Y)$ in $\mathcal{D}$.
2. F rispetta le identità: $F(\text{id}_X) = \text{id}_{F(X)}$ per ogni $X \in \mathcal{C}$.
3. F rispetta le composizioni: $F(g \circ f) = F(g) \circ F(f)$ per ogni morfismo $f : X \rightarrow Y$ e $g : Y \rightarrow Z$.

Per ogni categoria $\mathcal{C}$, è definito un *funtore identità* $\text{id}_{\mathcal{C}} : \mathcal{C} \rightarrow \mathcal{C}$, che associa $X \mapsto X$ e $f \mapsto f$. Dato un oggetto $A \in \mathcal{C}$, si definisce un *funtore costante* $\underline{A} : \mathcal{D} \rightarrow \mathcal{C}$, per una categoria arbitraria $\mathcal{D}$. Questo funtore associa ad ogni $X \in \mathcal{D}$ l'oggetto A e ad ogni morfismo $f \in \text{Mor}(\mathcal{D})$ il morfismo id_A .

Inoltre, dati due funtori $F : C \rightarrow D$ e $G : D \rightarrow E$, esiste un *functore composto* $G \circ F : C \rightarrow E$, che può anche essere indicato semplicemente come GF .

Un tipo di funtori particolarmente importante per lo studio delle coalgebre è dato dagli *endofuntori*, cioè funtori $F : C \rightarrow C$, che permettono di dare una definizione formale di coalgebra.

Definizione 2.4 (Prodotto e coprodotto). Dati due oggetti $X, Y \in C$:

1. Il **prodotto** $X \times Y$ è un oggetto dotato di morfismi $\pi_1 : X \times Y \rightarrow X$ e $\pi_2 : X \times Y \rightarrow Y$, detti *proiezioni*, tale che per ogni oggetto Z e morfismi $f : Z \rightarrow X$ e $g : Z \rightarrow Y$ esiste un unico morfismo $\langle f, g \rangle : Z \rightarrow X \times Y$ con $\pi_1 \circ \langle f, g \rangle = f$ e $\pi_2 \circ \langle f, g \rangle = g$:

$$\begin{array}{ccccc} & & Z & & \\ & f \swarrow & \downarrow \langle f, g \rangle & \searrow g & \\ X & \xleftarrow{\pi_1} & X \times Y & \xrightarrow{\pi_2} & Y \end{array}$$

2. Il **coprodotto** $X + Y$ è un oggetto dotato di morfismi $\kappa_1 : X \rightarrow X + Y$ e $\kappa_2 : Y \rightarrow X + Y$, detti *coproiezioni* o *inclusioni*, tale che per ogni oggetto Z e morfismi $f : X \rightarrow Z$ e $g : Y \rightarrow Z$ esiste un unico morfismo $[f, g] : X + Y \rightarrow Z$ con $[f, g] \circ \kappa_1 = f$ e $[f, g] \circ \kappa_2 = g$:

$$\begin{array}{ccccc} X & \xrightarrow{\kappa_1} & X + Y & \xleftarrow{\kappa_2} & Y \\ & \searrow f & \downarrow [f, g] & \swarrow g & \\ & & Z & & \end{array}$$

Nella categoria **Set** il prodotto è il prodotto cartesiano $X \times Y = \{(x, y) \mid x \in X, y \in Y\}$ con le proiezioni usuali, e il coprodotto è l'unione disgiunta $X + Y$. Entrambe le costruzioni si estendono ai morfismi: date $f : X \rightarrow X'$ e $g : Y \rightarrow Y'$, si definiscono $f \times g : X \times Y \rightarrow X' \times Y'$ tramite $(x, y) \mapsto (f(x), g(y))$ e $f + g : X + Y \rightarrow X' + Y'$ tramite le coproiezioni. Questo rende prodotto e coprodotto funtoriali [4].

Infine, una nozione categorica centrale nello studio delle coalgebre è quella di oggetto finale.

Definizione 2.5 (Oggetto finale). Un **oggetto finale** in una categoria C è un oggetto $\mathbf{1} \in C$ tale che per ogni oggetto $X \in C$ esiste un unico morfismo $!_X : X \rightarrow \mathbf{1}$.

Una proprietà importante degli oggetti finali è che sono unici a meno di un isomorfismo unico.

Proposizione 2.1. Se $\mathbf{1}$ e $\mathbf{1}'$ sono oggetti finali in una categoria C , allora esiste un isomorfismo unico $! : \mathbf{1} \rightarrow \mathbf{1}'$ tra essi.

Dimostrazione. Per la proprietà di oggetto finale, esistono morfismi unici $! : \mathbf{1} \rightarrow \mathbf{1}'$ e $! : \mathbf{1}' \rightarrow \mathbf{1}$. Considerando le loro composizioni, si costruiscono i seguenti diagrammi:

$$\mathbf{1} \xrightleftharpoons[!']{\quad} \mathbf{1}' \quad \Rightarrow \quad \begin{array}{c} \mathbf{1} \xrightarrow{!} \mathbf{1}' \\ \text{id}_1 \parallel \quad \xleftarrow{!'} \\ \mathbf{1} \end{array} \quad \text{e} \quad \begin{array}{c} \mathbf{1}' \xrightarrow{!'} \mathbf{1} \\ \text{id}_{1'} \parallel \quad \xleftarrow{!} \\ \mathbf{1}' \end{array}$$

Poiché il morfismo da un oggetto finale a se stesso è unico, la composizione dei due morfismi coincide con l'identità dell'oggetto di partenza:

$$!'\circ! = \text{id}_1 \quad \text{e} \quad !\circ!' = \text{id}_{1'}$$

Questo dimostra che $!$ è un isomorfismo (con inverso $!'$), e l'unicità è garantita dalla definizione di oggetto finale. $\square$

Esempio 2.2. Nella categoria **Set** qualunque insieme con un solo elemento è un oggetto finale: si sceglie canonicamente $\mathbf{1} = \{*\}$.

Definizione 2.6 (Relazione). Dati due oggetti $X, Y \in C$, una **relazione** R tra X e Y è un oggetto $R \in C$ dotato di un morfismo $\langle \pi_1, \pi_2 \rangle : R \rightarrow X \times Y$ le cui componenti $\pi_1 : R \rightarrow X$ e $\pi_2 : R \rightarrow Y$ sono dette *proiezioni* della relazione. Si scrive $R \subseteq X \times Y$ per indicare che R , tramite $\langle \pi_1, \pi_2 \rangle$, individua un sottoinsieme del prodotto $X \times Y$.

Nella categoria **Set** questa definizione coincide con la nozione insiemistica usuale: una relazione $R \subseteq X \times Y$ è un sottoinsieme del prodotto cartesiano, le cui proiezioni $\pi_1 : R \rightarrow X$ e $\pi_2 : R \rightarrow Y$ sono le restrizioni a R delle proiezioni di $X \times Y$. Dato $r = (x, y) \in R$, si scrive equivalentemente $x R y$. Questa nozione di relazione, formulata a livello di una categoria arbitraria, è quella che permetterà di definire in modo generale le bisimulazioni tra coalgebre [4, 5].

Definizione 2.7 (Esponenziale). Dati due oggetti $X, Y \in C$, l'**esponenziale** Y^X è un oggetto dotato di un morfismo di valutazione $\text{ev} : Y^X \times X \rightarrow Y$ tale che per ogni oggetto Z e ogni morfismo $g : Z \times X \rightarrow Y$ esiste un unico morfismo $\lambda(g) : Z \rightarrow Y^X$ che rende commutativo il seguente diagramma:

$$\begin{array}{ccc} Z \times X & \xrightarrow{\lambda(g) \times \text{id}_X} & Y^X \times X \\ & \searrow g & \downarrow \text{ev} \\ & & Y \end{array}$$

Nella categoria **Set** l'esponenziale Y^X è l'insieme di tutte le funzioni totali da X a Y , e il morfismo di valutazione è $\text{ev}(\varphi, x) = \varphi(x)$. La corrispondenza tra un morfismo $g : Z \times X \rightarrow Y$ e la sua *curryficazione* $\lambda(g) : Z \rightarrow Y^X$, definita da $\lambda(g)(z)(x) = g(z, x)$, è la nota corrispondenza biunivoca:

$$\frac{Z \times X \rightarrow Y}{Z \rightarrow Y^X}.$$

Questa costruzione permette di rappresentare una funzione di transizione con più argomenti, come $t : X \times A \rightarrow X$, tramite la sua forma currificata $t : X \rightarrow X^A$, dove $t(x)(a) \in X$ è lo

stato raggiunto da x leggendo il simbolo a . È in questa forma che le funzioni di transizione degli automi vengono espresse come componenti di una coalgebra [4].

3. Coalgebre

Definizione 3.1 (Coalgebra). Data una categoria C e un endofunttore $F : C \rightarrow C$:

1. Una **F -coalgebra** (si può omettere il funtore F se non crea confusione) consiste di un oggetto $X \in C$ e un morfismo $c : X \rightarrow F(X)$. X prende il nome di *spazio degli stati* o *insieme portante* della coalgebra, mentre c è detta *transizione* o *struttura* della coalgebra.
2. Un **omomorfismo di coalgebre** da una coalgebra $c : X \rightarrow F(X)$ a $d : Y \rightarrow F(Y)$ consiste di un morfismo $f : X \rightarrow Y$ che rende il diagramma seguente commutativo:

$$\begin{array}{ccc} F(X) & \xrightarrow{F(f)} & F(Y) \\ \uparrow c & & \uparrow d \\ X & \xrightarrow{f} & Y \end{array}$$

3. Le F -coalgebre con i loro omomorfismi formano una categoria **$\mathbf{CoAlg}(F)$** , che ammette un funtore **$\mathbf{CoAlg}(F) \rightarrow C$** che associa a ogni coalgebra $c : X \rightarrow F(X)$ il suo spazio degli stati X , e a ogni omomorfismo f lo stesso morfismo visto in C .

La commutatività del diagramma degli omomorfismi esprime una condizione di compatibilità: effettuare una transizione in X e poi applicare f produce lo stesso risultato che applicare f e poi effettuare una transizione in Y . Un omomorfismo di coalgebre è quindi una mappa che rispetta la dinamica di entrambi i sistemi.

L'endofunttore F è il parametro fondamentale della teoria: esso determina il *tipo di interfaccia* del sistema, cioè la struttura degli stati successivi e delle osservazioni accessibili. Per esempio, il funtore $\text{Seq}_A(X) = \{\perp\} \sqcup (A \times X)$ descrive sistemi che possono terminare o produrre un elemento di A con un successore [4].

3.1 Coalgebra finale e coricorsione

La nozione di oggetto finale, introdotta nel capitolo precedente, si specializza nella categoria **$\mathbf{CoAlg}(F)$** alla nozione di *coalgebra finale*.

Definizione 3.2 (Coalgebra finale). Una F -**coalgebra finale** è un oggetto finale nella categoria $\mathbf{CoAlg}(F)$, ovvero una coalgebra $\zeta : Z \rightarrow F(Z)$ tale che per ogni coalgebra $c : X \rightarrow F(X)$ esiste un unico omomorfismo di coalgre $\text{beh}_c : X \rightarrow Z$:

$$\begin{array}{ccc} F(X) & \xrightarrow{F(\text{beh}_c)} & F(Z) \\ \uparrow c & & \uparrow \zeta \\ X & \xrightarrow{\text{beh}_c} & Z \end{array}$$

Il morfismo beh_c è detto *funzione di comportamento*: essa mappa ogni stato $x \in X$ al suo *comportamento osservabile* come elemento di Z . Poiché la coalgebra finale è un oggetto finale in $\mathbf{CoAlg}(F)$, dalla proposizione del capitolo precedente segue che essa è unica a meno di isomorfismo unico.

La struttura di transizione di una coalgebra finale gode di una proprietà notevole: è essa stessa un isomorfismo.

Lemma 3.1 (Punto fisso di Lambek). Se $\zeta : Z \rightarrow F(Z)$ è una coalgebra finale per l'endofunore F , allora ζ è un isomorfismo, e in particolare $Z \cong F(Z)$.

Dimostrazione. Applicando F alla coalgebra finale $\zeta : Z \rightarrow F(Z)$ si ottiene una nuova coalgebra $F(\zeta) : F(Z) \rightarrow F(F(Z))$. Per finalità esiste un unico omomorfismo $f : F(Z) \rightarrow Z$ tale che il seguente diagramma commuti:

$$\begin{array}{ccc} F(F(Z)) & \xrightarrow{F(f)} & F(Z) \\ \uparrow F(\zeta) & & \uparrow \zeta \\ F(Z) & \xrightarrow{f} & Z \end{array}$$

Mostriamo che $f = \zeta^{-1}$. La composizione $f \circ \zeta : Z \rightarrow Z$ è un omomorfismo dalla coalgebra ζ a se stessa; anche id_Z lo è, e per unicità si conclude $f \circ \zeta = \text{id}_Z$. Per l'equazione inversa:

$$\zeta \circ f = F(f) \circ F(\zeta) = F(f \circ \zeta) = F(\text{id}_Z) = \text{id}_{F(Z)}.$$

Dunque $f = \zeta^{-1}$ e ζ è un isomorfismo. □

Il lemma dice che la coalgebra finale è un *punto fisso* dell'endofunore F : si ha $Z \cong F(Z)$. Per il funtore Seq_A questo diventa $A^\infty \cong \{\perp\} \sqcup (A \times A^\infty)$, l'equazione caratteristica delle sequenze finite e infinite [4].

La finalità fornisce un principio di **definizione per coricorsione**: per definire una funzione $h : X \rightarrow Z$ nella coalgebra finale è sufficiente dotare X di una struttura di F -coalgebra $c : X \rightarrow F(X)$, e la funzione di comportamento beh_c è l'unica funzione che si ottiene in questo modo. Non è necessario specificare h globalmente: è sufficiente descrivere come si comporta un passo alla volta, e la finalità garantisce l'esistenza e unicità della funzione definita.

3.2 Coinduzione come principio di dimostrazione

L'unicità del morfismo beh_c non è solo una proprietà tecnica: è il fondamento del **principio di coinduzione** [4, 5]. Nella sua forma più generale, esso afferma che per dimostrare l'uguaglianza di due funzioni $f, g : X \rightarrow Z$ verso la coalgebra finale è sufficiente esibire una struttura di coalgebra $c : X \rightarrow F(X)$ rispetto alla quale sia f che g siano omomorfismi, poiché l'unicità di beh_c ne forza allora la coincidenza.

3.3 Bisimulazione

Il principio di coinduzione afferma che due stati con la stessa funzione di comportamento sono indistinguibili. La *bisimulazione* è il concetto duale che cattura questa equivalenza comportamentale direttamente in termini di relazioni tra stati, senza fare riferimento alla coalgebra finale [5].

Definizione 3.3 (Bisimulazione). Siano $c : X \rightarrow F(X)$ e $d : Y \rightarrow F(Y)$ due F -coalgebre. Una **bisimulazione** tra c e d è una relazione $R \subseteq X \times Y$, con proiezioni $\pi_1 : R \rightarrow X$ e $\pi_2 : R \rightarrow Y$, tale che esista una struttura di F -coalgebra $e : R \rightarrow F(R)$ che rende π_1 e π_2 omomorfismi di coalgebre, cioè che rende commutativo il seguente diagramma:

$$\begin{array}{ccccc} F(X) & \xleftarrow{F(\pi_1)} & F(R) & \xrightarrow{F(\pi_2)} & F(Y) \\ \uparrow c & & \uparrow e & & \uparrow d \\ X & \xleftarrow{\pi_1} & R & \xrightarrow{\pi_2} & Y \end{array}$$

ovvero tale che esista $e : R \rightarrow F(R)$ con $F(\pi_1) \circ e = c \circ \pi_1$ e $F(\pi_2) \circ e = d \circ \pi_2$. Se $X = Y$ e $c = d$, si parla di bisimulazione sulla singola coalgebra c . Questa formulazione è nota come bisimulazione *alla Aczel–Mendler* [4].

La definizione afferma che $(x, y) \in R$ quando x e y si comportano allo stesso modo: ogni mossa di x può essere imitata da y restando in R , e viceversa. Il diagramma commutativo esprime che la relazione R , vista come coalgebra, è compatibile con entrambe le strutture di transizione tramite le proiezioni.

Proposizione 3.1. Le bisimulazioni godono delle seguenti proprietà di chiusura:

1. La relazione identità $\Delta_X = \{(x, x) \mid x \in X\}$ è una bisimulazione su ogni coalgebra $c : X \rightarrow F(X)$.
2. Se R è una bisimulazione tra c e d , allora la relazione inversa $R^{-1} = \{(y, x) \mid (x, y) \in R\}$ è una bisimulazione tra d e c .
3. Se R è una bisimulazione tra $c : X \rightarrow F(X)$ e $d : Y \rightarrow F(Y)$, e S è una bisimulazione tra d e $e : Z \rightarrow F(Z)$, allora la composizione relazionale $S \circ R = \{(x, z) \mid \exists y \in Y. (x, y) \in R \wedge (y, z) \in S\}$ è una bisimulazione tra c e e .

Dimostrazione. 1. La struttura di coalgebra su Δ_X è data da c stesso, poiché $\Delta_X \cong X$ tramite entrambe le proiezioni, che coincidono con id_X . La commutatività dei diagrammi segue immediatamente.

2. Se $e : R \rightarrow F(R)$ testimonia che R è una bisimulazione, la stessa struttura e su $R^{-1} \cong R$ testimonia che R^{-1} è una bisimulazione, scambiando i ruoli di π_1 e π_2 .

3. Siano $e_R : R \rightarrow F(R)$ e $e_S : S \rightarrow F(S)$ le strutture che testimoniano le due bisimulazioni. Su $S \circ R$ si definisce una struttura di coalgebra usando l'azione di F sulle proiezioni: poiché F preserva le composizioni e le proiezioni di $S \circ R$ si fattorizzano attraverso quelle di R e S , la commutatività dei diagrammi richiesti segue per funtorialità.

□

Le proprietà di chiusura mostrano che la *bisimilarità* — la relazione $\sim$ definita ponendo $x \sim y$ quando esiste almeno una bisimulazione che contiene la coppia (x, y) — è una relazione di equivalenza. Essa è inoltre la massima bisimulazione: l'unione di tutte le bisimulazioni è ancora una bisimulazione.

Il legame fondamentale tra bisimulazione e coinduzione è espresso dal seguente teorema.

Teorema 3.1. Sia $\zeta : Z \rightarrow F(Z)$ la coalgebra finale. Due stati $x, x' \in X$ di una coalgebra $c : X \rightarrow F(X)$ sono bisimili se e solo se hanno lo stesso comportamento:

$$x \sim x' \iff \text{beh}_c(x) = \text{beh}_c(x').$$

Dimostrazione. ($\Rightarrow$) Sia R una bisimulazione su c con $(x, x') \in R$. Allora R è essa stessa una coalgebra, e le proiezioni $\pi_1, \pi_2 : R \rightarrow X$ sono omomorfismi. Quindi $\text{beh}_c \circ \pi_1$ e $\text{beh}_c \circ \pi_2$ sono entrambi omomorfismi da R a ζ . Per unicità dell'omomorfismo verso la coalgebra finale, $\text{beh}_c \circ \pi_1 = \text{beh}_c \circ \pi_2$, cioè $\text{beh}_c(x) = \text{beh}_c(x')$.

($\Leftarrow$) Supponiamo $\text{beh}_c(x) = \text{beh}_c(x')$. La relazione $R = \{(u, v) \in X \times X \mid \text{beh}_c(u) = \text{beh}_c(v)\}$ è una bisimulazione: la struttura di coalgebra su R è data da $\langle c \circ \pi_1, c \circ \pi_2 \rangle$, e la commutatività dei diagrammi segue dal fatto che beh_c è un omomorfismo. Poiché $(x, x') \in R$, si conclude $x \sim x'$. □

Il teorema afferma che la bisimilarità e l'uguaglianza del comportamento osservabile sono la stessa cosa. Questo è il risultato fondamentale che permette di usare la bisimulazione come strumento pratico di dimostrazione: per mostrare che due stati si comportano allo stesso modo è sufficiente esibire una bisimulazione che li contenga, senza calcolare esplicitamente la loro funzione di comportamento [4, 5].

3.4 Predicati e invarianti

La teoria delle coalgebre fornisce anche un linguaggio per esprimere *proprietà* degli stati di un sistema, non solo la loro struttura di transizione [4]. Sia $c : X \rightarrow F(X)$ una F -coalgebra.

Per un sottoinsieme $P \subseteq X$, il *predicato successore* $\bigcirc_c P \subseteq X$ è definito come:

$$\bigcirc_c P = c^{-1}(F(P))$$

dove $F(P) \subseteq F(X)$ è l'immagine di P sotto l'azione del funtore F sull'inclusione $P \hookrightarrow X$, e c^{-1} denota l'immagine inversa lungo c . In parole: $\bigcirc_c P$ è l'insieme degli stati la cui transizione cade interamente in $F(P)$, ovvero degli stati i cui successori soddisfano tutti P .

Per il funtore $\text{Seq}_A : X \mapsto \{\perp\} \sqcup (A \times X)$ e una coalgebra $c : S \rightarrow \{\perp\} \sqcup (A \times S)$, si ha $F(P) = \{\perp\} \sqcup (A \times P)$, e quindi:

$$\bigcirc_c P = \{x \in S \mid c(x) = \perp \text{ oppure } c(x) = (a, x') \text{ con } x' \in P\}.$$

Usando la notazione di transizione $x \xrightarrow{a} x'$, questa condizione si legge: $\bigcirc_c P$ vale per x se tutti gli stati successori di x , qualora esistano, soddisfano P .

Definizione 3.4 (Invariante). Un predicato $P \subseteq X$ è un **invariante** della coalgebra $c : X \rightarrow F(X)$ se $P \subseteq \bigcirc_c P$, ovvero se $c(P) \subseteq F(P)$. Intuitivamente: una volta che P vale per uno stato, la sua transizione rimane in $F(P)$, ovvero tutti i successori soddisfano ancora P .

Proposizione 3.2. Gli invarianti godono delle seguenti proprietà di chiusura:

1. Il predicato costante vero, cioè X stesso, è un invariante.
2. Dati due invarianti P_1 e P_2 , la loro intersezione $P_1 \cap P_2$ è un invariante.
3. Dato un invariante P , anche $\bigcirc_c P$ è un invariante.

Dimostrazione. 1. Per ogni $x \in X$ si ha $c(x) \in F(X)$, quindi $X \subseteq \bigcirc_c X$.

2. Poiché F è un funtore preserva le inclusioni, quindi $F(P_1 \cap P_2) \subseteq F(P_1) \cap F(P_2)$.

Dunque:

$$\begin{aligned} x \in P_1 \cap P_2 &\implies x \in P_1 \wedge x \in P_2 \\ &\implies c(x) \in F(P_1) \wedge c(x) \in F(P_2) \quad (P_1, P_2 \text{ invarianti}) \\ &\implies c(x) \in F(P_1) \cap F(P_2) \\ &\implies c(x) \in F(P_1 \cap P_2) \\ &\implies x \in \bigcirc_c (P_1 \cap P_2). \end{aligned}$$

3. Bisogna verificare che $\bigcirc_c P \subseteq \bigcirc_c \bigcirc_c P$. Sia $x \in \bigcirc_c P$, cioè $c(x) \in F(P)$. Poiché P è un invariante, $c(P) \subseteq F(P)$, e quindi per funtorialità $F(c)(F(P)) \subseteq F(F(P))$. Poiché $P \subseteq \bigcirc_c P$ per invarianza, si ha $F(P) \subseteq F(\bigcirc_c P)$, e quindi $c(x) \in F(P) \subseteq F(\bigcirc_c P)$, cioè $x \in \bigcirc_c \bigcirc_c P$.

□

3.5 Operatori temporali

A partire dagli invarianti si definiscono i principali operatori temporali, in forma del tutto generale per una qualunque F -coalgebra [4].

Definizione 3.5 (Operatori temporali). Sia $c : X \rightarrow F(X)$ una F -coalgebra e $P \subseteq X$ un predicato arbitrario.

1. Il predicato $\Box P \subseteq X$ (*d'ora in poi P*) è il *massimo invariante contenuto in P*:

$$\Box P = \bigcup \{Q \subseteq X \mid Q \text{ è un invariante e } Q \subseteq P\}.$$

2. Il predicato $\Diamond P \subseteq X$ (*prima o poi P*) è definito come $\Diamond P = \neg \Box \neg P$.

Il predicato $\Box P$ vale per x quando tutti i futuri stati raggiungibili da x soddisfano P . Il predicato $\Diamond P$ vale quando almeno uno stato futuro soddisfa P .

Proposizione 3.3. L'operatore $\Box$ è un operatore di interno: per ogni predicato $P, Q \subseteq X$,

1. $\Box P \subseteq P$;
2. $\Box P \subseteq \Box \Box P$;
3. se $P \subseteq Q$ allora $\Box P \subseteq \Box Q$.

Inoltre, un predicato P è un invariante se e solo se $P = \Box P$.

Dimostrazione. 1. Per definizione, $\Box P$ è il massimo invariante contenuto in P , quindi in particolare $\Box P \subseteq P$.

2. Bisogna mostrare che ogni $x \in \Box P$ appartiene anche a $\Box \Box P$, cioè che $\Box P$ è un invariante contenuto in $\Box P$. È contenuto in $\Box P$ banalmente. È un invariante: per la proposizione precedente, $\Box P$ è un invariante (è il massimo invariante contenuto in P , e l'unione di invarianti è un invariante per chiusura sotto intersezioni arbitrarie). Poiché $\Box \Box P$ è per definizione il massimo invariante contenuto in $\Box P$, si conclude $\Box P \subseteq \Box \Box P$.

3. Sia $P \subseteq Q$. Ogni invariante R con $R \subseteq P$ soddisfa anche $R \subseteq Q$, e quindi è tra gli invarianti contenuti in Q . In particolare, $\Box P$ — che è un invariante contenuto in P , e quindi in Q — è uno degli invarianti la cui unione forma $\Box Q$. Dunque $\Box P \subseteq \Box Q$.

Per l'ultima affermazione. Se P è un invariante, allora P è esso stesso un invariante contenuto in P , quindi $P \subseteq \Box P$; d'altra parte $\Box P \subseteq P$ per il punto 1, quindi $P = \Box P$. Viceversa, se $P = \Box P$, allora P è un invariante perché $\Box P$ è per costruzione un'unione di invarianti, e la proposizione precedente garantisce che l'unione di invarianti è un invariante.

□

4. Esempi pratici di coalgebre in informatica teorica

4.1 Linguaggi regolari

Il primo esempio concreto di coalgebra applicata all'informatica teorica riguarda gli automi, di cui i riconoscitori di linguaggi regolari sono un caso particolare [4].

4.1.1 Il funtore degli automi

Siano A un insieme fissato, interpretato come *alfabeto di input*, e B un insieme fissato, interpretato come *insieme di output*. Si consideri l'endofuntore su **Set**:

$$F(X) = B \times X^A.$$

Una F -coalgebra $\langle o, \delta \rangle : X \rightarrow B \times X^A$ consiste di due componenti, ottenute componendo $\langle o, \delta \rangle$ con le proiezioni del prodotto:

- una funzione di **osservazione** (o output) $o : X \rightarrow B$, che associa a ogni stato $x \in X$ un valore osservabile $o(x) \in B$;
- una funzione di **transizione** $\delta : X \rightarrow X^A$, che, per curriificazione, corrisponde a una funzione $X \times A \rightarrow X$: dato uno stato x e un simbolo di input $a \in A$, $\delta(x)(a) \in X$ è lo stato successore raggiunto da x leggendo a .

Una tale coalgebra è detta **automa deterministico** con alfabeto di input A e output B , e X è il suo insieme di stati.

Un omomorfismo tra due automi $\langle o, \delta \rangle : X \rightarrow B \times X^A$ e $\langle o', \delta' \rangle : Y \rightarrow B \times Y^A$ è, per la definizione generale di omomorfismo di coalgebre, una funzione $f : X \rightarrow Y$ tale che:

$$o'(f(x)) = o(x) \quad \text{e} \quad f(\delta(x)(a)) = \delta'(f(x))(a)$$

per ogni stato $x \in X$ e ogni simbolo $a \in A$. La prima equazione dice che f preserva l'output; la seconda dice che f è compatibile con la transizione: applicare f e poi transitare in Y equivale a transitare in X e poi applicare f .

4.1.2 Automi come riconoscitori di linguaggi

Il caso di interesse per i linguaggi regolari si ottiene specializzando l'insieme di output a $B = 2 = \{0, 1\}$, dove 0 rappresenta uno stato non accettante e 1 uno stato accettante.

Definizione 4.1 (Funtore dei linguaggi). Sia A un alfabeto fissato. Il funtore $D_A : \mathbf{Set} \rightarrow \mathbf{Set}$ è definito da:

$$D_A(X) = 2 \times X^A.$$

Una D_A -coalgebra $\langle o, \delta \rangle : X \rightarrow 2 \times X^A$ è detta **automa deterministico riconoscitore** (o semplicemente automa) sull'alfabeto A : la componente $o : X \rightarrow 2$ indica se uno stato è accettante ($o(x) = 1$) o meno ($o(x) = 0$), mentre $\delta : X \rightarrow X^A$ è la funzione di transizione.

A differenza della nozione classica di automa a stati finiti [3, 6], questa definizione non richiede che X sia un insieme finito, né che venga fissato uno stato iniziale: la coalgebra descrive la dinamica dell'intero sistema di transizione. Uno stato iniziale, quando serve, si specifica separatamente scegliendo un elemento $x_0 \in X$; la coppia $(X, \langle o, \delta \rangle, x_0)$ corrisponde allora alla nozione usuale di automa a stati finiti con stato iniziale, nel caso in cui X sia finito.

Un omomorfismo di D_A -coalgebre tra $\langle o, \delta \rangle : X \rightarrow 2 \times X^A$ e $\langle o', \delta' \rangle : Y \rightarrow 2 \times Y^A$ è una funzione $f : X \rightarrow Y$ tale che $o = o' \circ f$ e $f(\delta(x)(a)) = \delta'(f(x))(a)$ per ogni $x \in X$ e $a \in A$: f preserva l'accettazione degli stati e rispetta le transizioni su ogni simbolo dell'alfabeto.

4.1.3 La coalgebra finale dei linguaggi

Per il funtore D_A , la coalgebra finale ha una descrizione concreta e familiare: è l'insieme di tutti i linguaggi su A [4, 5].

Proposizione 4.1. Sia A^* l'insieme delle parole finite su A , e sia $\mathcal{L}_A = 2^{A^*}$ l'insieme di tutti i linguaggi su A (cioè l'insieme delle parti di A^* , identificato con l'insieme delle funzioni $A^* \rightarrow 2$). La struttura

$$\langle o_L, \delta_L \rangle : \mathcal{L}_A \rightarrow 2 \times \mathcal{L}_A^A, \quad o_L(L) = \begin{cases} 1 & \text{se } \varepsilon \in L \\ 0 & \text{altrimenti} \end{cases}, \quad \delta_L(L)(a) = \{w \in A^* \mid aw \in L\}$$

è una coalgebra finale per il funtore D_A , dove ε denota la parola vuota. Questa struttura corrisponde all'operazione di scomposizione di un linguaggio rispetto al primo simbolo letto, studiata originariamente in un contesto puramente algebrico da [2].

La componente $\delta_L(L)(a) = \{w \in A^* \mid aw \in L\}$ è l'insieme delle parole che, precedute da a , appartengono ancora a L . La struttura di coalgebra su $\mathcal{L}_A$ codifica quindi due fatti elementari ma sufficienti a determinare completamente un linguaggio: se la parola vuota vi appartiene, e come si comporta il linguaggio dopo aver letto ciascun simbolo.

Per la finalit  di $\mathcal{L}_A$, ogni automa $\langle o, \delta \rangle : X \rightarrow 2 \times X^A$ ammette un unico omomorfismo $\text{beh} : X \rightarrow \mathcal{L}_A$, la funzione di comportamento. Esplicitando la condizione di omomorfismo

rispetto alla struttura $\langle o_L, \delta_L \rangle$, si ottiene che beh è l'unica funzione che soddisfa:

$$\varepsilon \in \text{beh}(x) \iff o(x) = 1, \quad \delta_L(\text{beh}(x))(a) = \text{beh}(\delta(x)(a))$$

per ogni stato $x \in X$ e ogni simbolo $a \in A$. Si può verificare che questa funzione è data esplicitamente da:

$$\text{beh}(x) = \{w \in A^* \mid \hat{\delta}(x, w) \text{ è uno stato accettante}\},$$

dove $\hat{\delta} : X \times A^* \rightarrow X$ è l'estensione di δ alle parole, definita ricorsivamente da $\hat{\delta}(x, \varepsilon) = x$ e $\hat{\delta}(x, aw) = \hat{\delta}(\delta(x)(a), w)$. In altre parole, $\text{beh}(x)$ è precisamente il **linguaggio riconosciuto** dall'automa a partire dallo stato x : l'insieme di tutte le parole che, lette a partire da x , conducono a uno stato accettante.

Questa osservazione dà un significato preciso e generale alla nozione di linguaggio riconosciuto da un automa: non è altro che il comportamento coalgebrico dello stato nella coalgebra finale di D_A . Un linguaggio $L \subseteq A^*$ è **regolare** se è il comportamento di uno stato di un automa $\langle o, \delta \rangle : X \rightarrow 2 \times X^A$ con X finito; equivalentemente, se $L = \text{beh}(x_0)$ per uno stato iniziale x_0 di un automa a stati finiti nel senso classico.

4.1.4 Chiusura dei linguaggi regolari

La descrizione coalgebrica permette di dimostrare le proprietà di chiusura dei linguaggi regolari [3, 6] costruendo direttamente automi (cioè D_A -coalgebre) per il complemento, l'unione, l'intersezione, la concatenazione e la stella di Kleene, e verificandone la correttezza esibendo esplicitamente un omomorfismo tra il nuovo automa e la coalgebra finale dei linguaggi.

Proposizione 4.2 (Chiusura per complemento). Se $L \subseteq A^*$ è regolare, anche il suo complemento $A^* \setminus L$ è regolare.

Dimostrazione. Sia $\langle o, \delta \rangle : X \rightarrow 2 \times X^A$ un automa finito con stato x_0 tale che $\text{beh}(x_0) = L$. Si definisce l'automa $\langle \bar{o}, \delta \rangle : X \rightarrow 2 \times X^A$ sullo stesso insieme di stati X e con la stessa transizione δ , ma con osservazione complementata $\bar{o}(x) = \overline{o(x)}$.

Si consideri la funzione $\overline{\text{beh}} : X \rightarrow \mathcal{L}_A$ definita da $\overline{\text{beh}}(x) = A^* \setminus \text{beh}(x)$, e si verifichi che è un omomorfismo da $\langle \bar{o}, \delta \rangle$ a $\langle o_L, \delta_L \rangle$:

$$\varepsilon \in \overline{\text{beh}}(x) \iff \varepsilon \notin \text{beh}(x) \iff o(x) = 0 \iff \bar{o}(x) = 1,$$

e per la transizione, usando il fatto che $\delta_L(A^* \setminus K)(a) = A^* \setminus \delta_L(K)(a)$ per ogni linguaggio

K (poiché una parola w soddisfa $aw \notin K$ se e solo se $aw \in A^* \setminus K$):

$$\begin{aligned}\delta_L(\overline{\text{beh}(x)})(a) &= \delta_L(A^* \setminus \text{beh}(x))(a) \\ &= A^* \setminus \delta_L(\text{beh}(x))(a) \\ &= A^* \setminus \text{beh}(\delta(x)(a)) \\ &= \overline{\text{beh}(\delta(x)(a))}.\end{aligned}$$

Poiché $\overline{\text{beh}}$ è un omomorfismo verso la coalgebra finale, per unicità coincide con la funzione di comportamento dell'automa $\langle \bar{o}, \delta \rangle$: $\overline{\text{beh}}(x) = A^* \setminus \text{beh}(x)$ per ogni $x \in X$, e in particolare per x_0 si ottiene $A^* \setminus L$. Poiché X è finito, $A^* \setminus L$ è regolare. $\square$

Le proprietà di chiusura per unione e intersezione si dimostrano in modo del tutto analogo, costruendo un automa *prodotto*.

Proposizione 4.3 (Chiusura per unione e intersezione). Se $L_1, L_2 \subseteq A^*$ sono regolari, anche $L_1 \cup L_2$ e $L_1 \cap L_2$ sono regolari.

Dimostrazione. Siano $\langle o_1, \delta_1 \rangle : X_1 \rightarrow 2 \times X_1^A$ e $\langle o_2, \delta_2 \rangle : X_2 \rightarrow 2 \times X_2^A$ automi finiti con stati iniziali x_1^0, x_2^0 tali che $\text{beh}_1(x_1^0) = L_1$ e $\text{beh}_2(x_2^0) = L_2$. Si costruisce l'automa *prodotto* sull'insieme di stati $X_1 \times X_2$, con transizione $\delta_\times : X_1 \times X_2 \rightarrow (X_1 \times X_2)^A$ data da:

$$\delta_\times(x_1, x_2)(a) = (\delta_1(x_1)(a), \delta_2(x_2)(a)),$$

e osservazioni, per l'unione e intersezione rispettivamente, date da:

$$o_\vee(x_1, x_2) = o_1(x_1) \vee o_2(x_2)$$

$$o_\wedge(x_1, x_2) = o_1(x_1) \wedge o_2(x_2)$$

Sia $g_\vee : X_1 \times X_2 \rightarrow \mathcal{L}_A$ definita da $g_\vee(x_1, x_2) = \text{beh}_1(x_1) \cup \text{beh}_2(x_2)$. Si verifica che $g_\vee$ è un omomorfismo da $\langle o_\vee, \delta_\times \rangle$ a $\langle o_L, \delta_L \rangle$:

$$\begin{aligned}\varepsilon \in g_\vee(x_1, x_2) &\iff \varepsilon \in \text{beh}_1(x_1) \vee \varepsilon \in \text{beh}_2(x_2) \\ &\iff o_1(x_1) = 1 \vee o_2(x_2) = 1 \\ &\iff o_\vee(x_1, x_2) = 1,\end{aligned}$$

e per la transizione, usando il fatto che $\delta_L(K_1 \cup K_2)(a) = \delta_L(K_1)(a) \cup \delta_L(K_2)(a)$ per ogni coppia di linguaggi K_1, K_2 (poiché una parola w soddisfa $aw \in K_1 \cup K_2$ se e solo se $aw \in K_1$ oppure $aw \in K_2$):

$$\begin{aligned}\delta_L(g_\vee(x_1, x_2))(a) &= \delta_L(\text{beh}_1(x_1))(a) \cup \delta_L(\text{beh}_2(x_2))(a) \\ &= \text{beh}_1(\delta_1(x_1)(a)) \cup \text{beh}_2(\delta_2(x_2)(a)) \\ &= g_\vee(\delta_\times(x_1, x_2)(a)).\end{aligned}$$

Per unicità dell'omomorfismo verso la coalgebra finale, g_v coincide con la funzione di comportamento dell'automa $\langle o_v, \delta_x \rangle$, e dunque $g_v(x_1^0, x_2^0) = L_1 \cup L_2$ è il comportamento dello stato iniziale. Poiché $X_1 \times X_2$ è finito, $L_1 \cup L_2$ è regolare.

La dimostrazione per l'intersezione è identica, sostituendo o_v con $o_\wedge$, g_v con $g_\wedge(x_1, x_2) = \text{beh}_1(x_1) \cap \text{beh}_2(x_2)$, e $\cup$ con $\cap$ in ogni passaggio. $\square$

Le tre dimostrazioni condividono la stessa struttura, che è il punto di forza dell'approccio coalgebrico: si costruisce un nuovo automa a partire da quelli dati, componendo transizione e osservazione componente per componente, si propone una funzione candidata verso la coalgebra finale espressa in termini dei comportamenti di partenza, e si verifica che tale funzione sia un omomorfismo. L'unicità garantita dalla finalità identifica allora questa funzione con il vero comportamento del nuovo automa. La finitezza dello spazio degli stati, che garantisce la regolarità del linguaggio risultante, si preserva automaticamente: il prodotto di due insiemi finiti è finito, e lo stesso insieme di stati basta per il complemento.

Le rimanenti operazioni booleane sulla classe dei linguaggi regolari, come la differenza simmetrica o l'implicazione, si ottengono componendo le tre costruzioni presentate, poiché ogni funzione booleana si esprime a partire da complemento, unione e intersezione.

4.1.5 Chiusura per concatenazione e stella di Kleene

Le due operazioni fondamentali che restano da trattare, la concatenazione e la stella di Kleene, non si lasciano esprimere come semplice composizione dei due automi di partenza: la difficoltà è che, letta una parola $w = uv$ con $u \in L_1$, l'automa deve "passare" alla lettura di v secondo L_2 , ma il punto di transizione tra u e v non è noto a priori e può non essere unico. Si risolve il problema tracciando, ad ogni istante, non un singolo stato di L_2 ma l'insieme di tutti gli stati di L_2 raggiungibili secondo le decomposizioni di w lette fino a quel punto, secondo la classica costruzione per sottoinsiemi [3].

Proposizione 4.4 (Chiusura per concatenazione). Se $L_1, L_2 \subseteq A^*$ sono regolari, anche $L_1 \cdot L_2 = \{uv \mid u \in L_1, v \in L_2\}$ è regolare.

Dimostrazione. Siano $\langle o_1, \delta_1 \rangle : X_1 \rightarrow 2 \times X_1^A$ e $\langle o_2, \delta_2 \rangle : X_2 \rightarrow 2 \times X_2^A$ automi finiti con stati iniziali x_1^0, x_2^0 tali che $\text{beh}_1(x_1^0) = L_1$ e $\text{beh}_2(x_2^0) = L_2$. Si costruisce l'automa $\langle o_\bullet, \delta_\bullet \rangle$ sull'insieme di stati $Y = X_1 \times \mathcal{P}(X_2)$, definito da:

$$\delta_\bullet(x_1, S)(a) = (\delta_1(x_1)(a), S_a), \quad S_a = \delta_2(S)(a) \cup \begin{cases} \{x_2^0\} & \text{se } o_1(\delta_1(x_1)(a)) = 1, \\ \emptyset & \text{se } o_1(\delta_1(x_1)(a)) = 0, \end{cases}$$

dove $\delta_2(S)(a) = \{\delta_2(s)(a) \mid s \in S\}$, e con osservazione $o_\bullet(x_1, S) = 1$ se e solo se esiste $s \in S$ con $o_2(s) = 1$. Lo stato iniziale è (x_1^0, S_0) , dove $S_0 = \{x_2^0\}$ se $o_1(x_1^0) = 1$, e $S_0 = \emptyset$ se $o_1(x_1^0) = 0$.

Intuitivamente, l'insieme S accumula lo stato di L_2 per ogni possibile decomposizione già completata: ogni volta che la componente X_1 raggiunge uno stato accettante, si aggiunge una nuova copia di x_2^0 a S , che da quel momento evolve parallelamente secondo δ_2 .

Sia $g : Y \rightarrow \mathcal{L}_A$ definita da $g(x_1, S) = \text{beh}_1(x_1) \cdot L_2 \cup \bigcup_{s \in S} \text{beh}_2(s)$. Si nota preliminarmente che, per costruzione di $\delta_\bullet$ e S_0 , vale l'invariante seguente su ogni stato (x_1, S) raggiungibile da (x_1^0, S_0) :

$$o_1(x_1) = 1 \implies x_2^0 \in S.$$

Infatti l'invariante vale per lo stato iniziale per costruzione di S_0 , ed è preservato da $\delta_\bullet$: se $o_1(\delta_1(x_1)(a)) = 1$, allora $x_2^0 \in S_a$ per costruzione.

Si verifica ora che g è un omomorfismo da $\langle o_\bullet, \delta_\bullet \rangle$ a $\langle o_L, \delta_L \rangle$. Per l'osservazione, $\varepsilon \in g(x_1, S)$ se e solo se $\varepsilon \in \text{beh}_1(x_1) \cdot L_2$ oppure $\varepsilon \in \text{beh}_2(s)$ per qualche $s \in S$; la prima condizione equivale a $o_1(x_1) = 1$ e $o_2(x_2^0) = 1$, e per l'invariante ciò implica $x_2^0 \in S$, rendendo questa condizione un caso particolare della seconda. Dunque:

$$\varepsilon \in g(x_1, S) \iff \exists s \in S. o_2(s) = 1 \iff o_\bullet(x_1, S) = 1.$$

Per la transizione, si calcola innanzitutto $\delta_L(K \cdot L_2)(a)$ per un linguaggio arbitrario K , direttamente dalla definizione $\delta_L(K \cdot L_2)(a) = \{w \in A^* \mid aw \in K \cdot L_2\}$. Una parola aw appartiene a $K \cdot L_2$ se e solo se si decompone come $aw = uv$ con $u \in K$ e $v \in L_2$. Se $u \neq \varepsilon$, allora u inizia necessariamente con a , cioè $u = au'$ con $u' \in \delta_L(K)(a)$, e $w = u'v$ con $v \in L_2$, dando un contributo $\delta_L(K)(a) \cdot L_2$. Se invece $u = \varepsilon$, possibile solo se $\varepsilon \in K$, allora $v = aw$, cioè $w \in \delta_L(L_2)(a)$. Riassumendo:

$$\delta_L(K \cdot L_2)(a) = \delta_L(K)(a) \cdot L_2 \cup T_K, \quad \text{dove } T_K = \begin{cases} \delta_L(L_2)(a) & \text{se } \varepsilon \in K, \\ \emptyset & \text{altrimenti.} \end{cases}$$

Si distinguono ora due casi a seconda del valore di $o_1(\delta_1(x_1)(a))$. Se $o_1(\delta_1(x_1)(a)) = 0$, allora $S_a = \delta_2(S)(a)$, e usando l'identità appena calcolata con $K = \text{beh}_1(x_1)$ (per cui $\varepsilon \notin \text{beh}_1(x_1)$, altrimenti $o_1(x_1) = 1$ e, per l'invariante, $x_2^0 \in S$, ma il caso generale è comunque coperto dal conto seguente) si ottiene:

$$\begin{aligned} \delta_L(g(x_1, S))(a) &= \delta_L(\text{beh}_1(x_1))(a) \cdot L_2 \cup \bigcup_{s \in S} \delta_L(\text{beh}_2(s))(a) \\ &= \text{beh}_1(\delta_1(x_1)(a)) \cdot L_2 \cup \bigcup_{s \in S} \text{beh}_2(\delta_2(s)(a)) \\ &= g(\delta_1(x_1)(a), S_a). \end{aligned}$$

Se invece $o_1(\delta_1(x_1)(a)) = 1$, allora $S_a = \delta_2(S)(a) \cup \{x_2^0\}$, e poiché $\varepsilon \in \text{beh}_1(\delta_1(x_1)(a))$

si ha $L_2 \subseteq \text{beh}_1(\delta_1(x_1)(a)) \cdot L_2$. Dunque:

$$\begin{aligned} g(\delta_1(x_1)(a), S_a) &= \text{beh}_1(\delta_1(x_1)(a)) \cdot L_2 \cup \bigcup_{s \in S} \text{beh}_2(\delta_2(s)(a)) \cup \text{beh}_2(x_2^0) \\ &= \text{beh}_1(\delta_1(x_1)(a)) \cdot L_2 \cup \bigcup_{s \in S} \text{beh}_2(\delta_2(s)(a)), \end{aligned}$$

poiché $\text{beh}_2(x_2^0) = L_2 \subseteq \text{beh}_1(\delta_1(x_1)(a)) \cdot L_2$ è assorbito nel primo termine. Questa espressione coincide con $\delta_L(g(x_1, S))(a)$, calcolata come nel caso precedente ma includendo ora anche il contributo $T_{\text{beh}_1(x_1)}$ quando $\varepsilon \in \text{beh}_1(x_1)$: tale contributo, per l'invariante, è già assorbito nell'unione su S poiché $x_2^0 \in S$. L'omomorfismo è dunque verificato anche in questo caso.

Per unicità dell'omomorfismo verso la coalgebra finale, g coincide con la funzione di comportamento dell'automa $\langle o_\bullet, \delta_\bullet \rangle$. In particolare:

$$g(x_1^0, S_0) = \text{beh}_1(x_1^0) \cdot L_2 \cup \bigcup_{s \in S_0} \text{beh}_2(s) = L_1 \cdot L_2,$$

dove l'ultima uguaglianza segue perché, se $S_0 = \{x_2^0\}$ (cioè $\varepsilon \in L_1$), il termine aggiuntivo $\text{beh}_2(x_2^0) = L_2$ è già contenuto in $L_1 \cdot L_2$ tramite la decomposizione con $u = \varepsilon$. Poiché $Y = X_1 \times \mathcal{P}(X_2)$ è finito, $L_1 \cdot L_2$ è regolare. $\square$

Una costruzione analoga, applicata a un singolo automa, produce la chiusura per stella di Kleene.

Proposizione 4.5 (Chiusura per stella di Kleene). Se $L \subseteq A^*$ è regolare, anche $L^* = \bigcup_{n \geq 0} L^n$ è regolare.

Dimostrazione. Sia $\langle o, \delta \rangle : X \rightarrow 2 \times X^A$ un automa finito con stato iniziale x_0 tale che $\text{beh}(x_0) = L$. Si costruisce l'automa $\langle o_*, \delta_* \rangle$ sull'insieme di stati $Y = \mathcal{P}(X)$, definito da:

$$\delta_*(S)(a) = \delta(S)(a) \cup \begin{cases} \{\delta(x_0)(a)\} & \text{se } \exists s \in S. o(s) = 1, \\ \emptyset & \text{altrimenti,} \end{cases}$$

$$o_*(S) = \begin{cases} 1 & \text{se } S = \{x_0\} \text{ oppure } \exists s \in S. o(s) = 1, \\ 0 & \text{altrimenti,} \end{cases}$$

con stato iniziale $S_0 = \{x_0\}$. Intuitivamente, ogni volta che uno stato attivo $s \in S$ è accettante per L (cioè una copia di L si è appena conclusa), si aggiunge al nuovo insieme di stati anche $\delta(x_0)(a)$, simulando l'avvio immediato di una nuova copia; lo stato $\{x_0\}$ è dichiarato sempre accettante, poiché rappresenta il punto in cui un numero qualsiasi (anche nullo) di copie di L si è concluso esattamente in quel punto.

Si definisce $g : Y \rightarrow \mathcal{L}_A$ da:

$$g(S) = \left(\bigcup_{s \in S} \text{beh}(s) \cdot L^* \right) \cup \begin{cases} \{\varepsilon\} & \text{se } S = \{x_0\}, \\ \emptyset & \text{altrimenti.} \end{cases}$$

Per l'osservazione, $\varepsilon \in g(S)$ se e solo se $S = \{x_0\}$, oppure $\varepsilon \in \text{beh}(s) \cdot L^*$ per qualche $s \in S$ (cioè $o(s) = 1$, poiché $\varepsilon \in L^*$ sempre); questa è esattamente la condizione $o_*(S) = 1$.

Per la transizione, si calcola innanzitutto $\delta_L(L^*)(a)$ direttamente dalla sua definizione. Una parola aw appartiene a L^* se e solo se si decompone in un numero finito di fattori non vuoti ciascuno appartenente a L (i fattori vuoti si possono sempre eliminare senza cambiare la decomposizione). Il primo fattore, essendo non vuoto, inizia con a ; scrivendolo come $au' \in L$, cioè $u' \in \delta_L(L)(a) = \text{beh}(\delta(x_0)(a))$, la parola restante appartiene di nuovo a L^* . Dunque:

$$\delta_L(L^*)(a) = \text{beh}(\delta(x_0)(a)) \cdot L^*.$$

Per un generico stato $s \in X$, applicando alla coppia $K = \text{beh}(s)$ lo stesso calcolo svolto per la concatenazione (Proposizione precedente), si ottiene:

$$\delta_L(\text{beh}(s) \cdot L^*)(a) = \text{beh}(\delta(s)(a)) \cdot L^* \cup T_s, \quad T_s = \begin{cases} \text{beh}(\delta(x_0)(a)) \cdot L^* & \text{se } o(s) = 1, \\ \emptyset & \text{altrimenti.} \end{cases}$$

dove si è usata l'identità appena ricavata per $\delta_L(L^*)(a)$. Sommando su tutti gli $s \in S$ (il contributo di $\{\varepsilon\}$, quando presente, si annulla poiché $\delta_L(\{\varepsilon\})(a) = \emptyset$):

$$\delta_L(g(S))(a) = \bigcup_{s \in S} \text{beh}(\delta(s)(a)) \cdot L^* \cup \bigcup_{s \in S, o(s)=1} \text{beh}(\delta(x_0)(a)) \cdot L^*.$$

Il secondo termine è non vuoto esattamente quando $\exists s \in S. o(s) = 1$, condizione che coincide con quella che governa l'aggiunta di $\delta(x_0)(a)$ a $\delta_*(S)(a)$. Dunque:

$$\delta_L(g(S))(a) = \bigcup_{s' \in \delta_*(S)(a)} \text{beh}(s') \cdot L^*.$$

Questa espressione coincide con $g(\delta_*(S)(a))$, a meno dell'eventuale termine $\{\varepsilon\}$ che comparirebbe in $g(\delta_*(S)(a))$ se $\delta_*(S)(a) = \{x_0\}$: in tal caso, però, $x_0 \in \delta_*(S)(a)$ implica che il termine $\text{beh}(x_0) \cdot L^* = L \cdot L^*$ è già presente nell'unione, e poiché $L^* = \{\varepsilon\} \cup L \cdot L^*$ per definizione di stella di Kleene, si ha $\varepsilon \notin L \cdot L^*$ solo se $\varepsilon \notin L$; ma in tal caso l'unione ristretta al solo stato x_0 resta comunque priva di ε , consistentemente con il fatto che $\delta_L(g(S))(a)$ sopra calcolato non contiene ε in quel caso. L'omomorfismo è dunque verificato.

Per unicità dell'omomorfismo verso la coalgebra finale, g coincide con la funzione di comportamento dell'automa $\langle o_*, \delta_* \rangle$, e in particolare:

$$g(S_0) = g(\{x_0\}) = \text{beh}(x_0) \cdot L^* \cup \{\varepsilon\} = L \cdot L^* \cup \{\varepsilon\} = L^*.$$

Poiché $Y = \mathcal{P}(X)$ è finito, L^* è regolare. □

Anche in questo caso la dimostrazione segue lo schema generale: costruire un nuovo automa su uno spazio di stati finito ottenuto componendo o iterando gli spazi di partenza tramite prodotto e insieme delle parti, proporre una funzione candidata verso la coalgebra finale, e verificarne l'omomorfismo esplicitando le due condizioni della definizione. L'unicità garantita dalla finalit  di $\mathcal{L}_A$ identifica allora questa funzione con il vero comportamento del nuovo automa.

4.2 Grammatiche libere dal contesto

Il secondo esempio riguarda le grammatiche libere dal contesto (*context-free grammar*, CFG) [3, 1], che a differenza degli automi non descrivono una singola sequenza di transizioni ma una *ramificazione*: ogni simbolo non terminale pu  espandersi, secondo una o pi  produzioni, in una sequenza di simboli successivi. Questa ramificazione richiede un funtore diverso da D_A .

4.2.1 Il funtore degli alberi di derivazione

Sia Σ un insieme fissato, interpretato come alfabeto dei simboli terminali. Per ogni insieme X , sia $X^* = \bigcup_{n \geq 0} X^n$ l'insieme delle sequenze finite di elementi di X (incluso il prodotto vuoto per $n = 0$, che ha come unico elemento la sequenza vuota). Si consideri l'endofuntore su **Set**:

$$F(X) = \Sigma + \mathcal{P}(X^*).$$

Una F -coalgebra $e : X \rightarrow \Sigma + \mathcal{P}(X^*)$ descrive un sistema in cui ogni elemento $x \in X$   di due tipi possibili:

- se $e(x) = \kappa_1(a)$ per qualche $a \in \Sigma$, allora x   un simbolo **terminale**: non si espande ulteriormente;
- se $e(x) = \kappa_2(R)$ per un insieme $R \subseteq X^*$, allora x   un simbolo **non terminale**, ed R   l'insieme delle sue possibili **espansioni**: ciascun elemento di R   una sequenza finita $(x_1, \dots, x_k) \in X^*$ di successori, corrispondente a una produzione applicabile a x .

La presenza dell'insieme delle parti $\mathcal{P}$ rende questa coalgebra *non deterministica*: a un simbolo non terminale possono corrispondere pi  produzioni alternative, esattamente come per gli automi non deterministici gi  incontrati nel capitolo precedente.

Un omomorfismo tra due coalgrebre $e : X \rightarrow \Sigma + \mathcal{P}(X^*)$ e $e' : Y \rightarrow \Sigma + \mathcal{P}(Y^*)$   una funzione $f : X \rightarrow Y$ che rispetta questa struttura: se x   terminale con $e(x) = \kappa_1(a)$, allora $e'(f(x)) = \kappa_1(a)$; se x   non terminale con $e(x) = \kappa_2(R)$, allora $e'(f(x)) = \kappa_2(R')$, dove $R' = \{(f(x_1), \dots, f(x_k)) \mid (x_1, \dots, x_k) \in R\}$   l'immagine di R ottenuta applicando f componente per componente a ciascuna sequenza.

4.2.2 Grammatiche come coalgebre

Una grammatica libera dal contesto $G = (N, \Sigma, P, S)$, con N insieme dei simboli non terminali, Σ alfabeto dei simboli terminali, P insieme delle produzioni della forma $A \rightarrow \alpha$ con $A \in N$ e $\alpha \in (N \cup \Sigma)^*$, e $S \in N$ simbolo iniziale, induce in modo canonico una F -coalgebra sull'insieme $X = N \cup \Sigma$ di tutti i suoi simboli:

$$e_G(x) = \begin{cases} \kappa_1(x) & \text{se } x \in \Sigma, \\ \kappa_2(\{\alpha \mid (x \rightarrow \alpha) \in P\}) & \text{se } x \in N. \end{cases}$$

In parole: ogni simbolo terminale è coalgebricamente terminale nel senso della definizione precedente; ogni simbolo non terminale A ha come espansioni esattamente i membri destri di tutte le produzioni di A presenti nella grammatica. Il simbolo iniziale $S \in X$ individua il punto di partenza da cui si originano tutte le derivazioni descritte dalla grammatica.

Esempio 4.1. Si consideri la grammatica per le espressioni aritmetiche ben parentesizzate su $\{+, (,)\} \cup \{n\}$, con un unico non terminale E e produzioni $E \rightarrow n \mid E + E \mid (E)$. La coalgebra corrispondente ha $e_G(E) = \kappa_2(\{n, E + E, (E)\})$, mentre ogni simbolo terminale $a \in \{n, +, (,)\}$ soddisfa $e_G(a) = \kappa_1(a)$.

Diversamente dal caso degli automi, questa coalgebra non ha una nozione altrettanto immediata di coalgebra finale che restituisca “il linguaggio generato”: la struttura e_G descrive direttamente l'insieme delle produzioni disponibili a ogni simbolo, ed è già essa stessa l'oggetto di interesse primario. È su questa struttura, piuttosto che sul suo comportamento in una coalgebra finale, che si applicano gli operatori temporali sviluppati nel capitolo precedente [4].

4.2.3 Proprietà strutturali tramite operatori temporali

Applicando la definizione generale del capitolo precedente, il predicato successore $\bigcirc_G P \subseteq X$ per un sottoinsieme $P \subseteq X$ si specializza, per il funtore $F(X) = \Sigma + \mathcal{P}(X^*)$, come segue. Poiché la componente Σ di F è costante (non dipende da X), ogni simbolo terminale soddisfa banalmente $\bigcirc_G P$, qualunque sia P . Per un simbolo non terminale A con $e_G(A) = \kappa_2(R)$, invece:

$$A \in \bigcirc_G P \iff \forall (A \rightarrow \alpha) \in R. \forall x \in \alpha. x \in P,$$

cioè: A soddisfa $\bigcirc_G P$ se e solo se *ogni* produzione di A contiene, nel membro destro, solamente simboli di P . Si noti la doppia quantificazione universale, su tutte le produzioni e su tutti i simboli di ciascuna produzione: riflette il fatto che, non essendoci alcuna scelta di produzione privilegiata, un simbolo soddisfa $\bigcirc_G P$ solo se P è preservato indipendentemente da quale produzione venga applicata.

Proposizione 4.6 (Contenimento sintattico). Sia $P \subseteq X$ un insieme di simboli. Se $x \in \Box P$, allora ogni simbolo che compare in un qualunque albero di derivazione con radice x , ottenuto applicando ripetutamente le produzioni di G in un modo qualsiasi, appartiene a P .

Dimostrazione. Per definizione, $\Box P$ è il massimo invariante contenuto in P , dunque in particolare è esso stesso un invariante: $\Box P \subseteq \bigcirc_G \Box P$. Sia $x \in \Box P$ e sia $x_1, \dots, x_k$ una qualunque sequenza di simboli ottenuta espandendo x tramite una produzione applicabile. Per la caratterizzazione di $\bigcirc_G$ sopra, $x \in \bigcirc_G \Box P$ implica $x_i \in \Box P$ per ogni i , e in particolare $x_i \in P$ per il punto 1 della proposizione sull'operatore di interno ($\Box P \subseteq P$). Iterando questo argomento su ogni livello dell'albero di derivazione, cioè procedendo per induzione sulla profondità del nodo nell'albero, usando a ogni passo che il simbolo corrente appartiene a $\Box P$, quindi a P , e i suoi successori appartengono di nuovo a $\Box P$, si conclude che ogni simbolo dell'albero appartiene a P . $\square$

Questa proposizione dà un significato preciso alla nozione di “contenimento sintattico” di una grammatica: prendendo P come il complemento di un simbolo indesiderato $z \in X$, $S \in \Box P$ esprime che z non compare in *nessun* albero di derivazione originato da S , qualunque sequenza di produzioni si scelga di applicare — una proprietà di sicurezza nel senso già discusso nel capitolo precedente, qui applicata all'analisi strutturale della grammatica anziché alla dinamica di un automa.

Proposizione 4.7 (Raggiungibilità). Sia $P \subseteq X$ un insieme di simboli, e sia $x \in X$. Se esiste un albero di derivazione con radice x che contiene, tra i suoi nodi, un simbolo appartenente a P , allora $x \in \Diamond P$.

Dimostrazione. Si consideri l'insieme

$$U = \{x \in X \mid \text{nessun albero di derivazione con radice } x \text{ contiene un simbolo di } P\}.$$

Si mostra che U è un invariante contenuto in $X \setminus P$, da cui la tesi seguirà per massimalità di $\Box$.

Anzitutto $U \subseteq X \setminus P$: se $x \in P$, l'albero banale costituito dal solo nodo x è un albero di derivazione con radice x che contiene un simbolo di P (x stesso), quindi $x \notin U$.

Si mostra ora che U è un invariante, cioè $U \subseteq \bigcirc_G U$. Sia $x \in U$ non terminale (il caso terminale è immediato, poiché ogni simbolo terminale soddisfa banalmente $\bigcirc_G U$), e sia $(x_1, \dots, x_k)$ una qualunque sequenza ottenuta da una produzione applicabile a x . Se, per assurdo, qualche $x_i \notin U$, allora esisterebbe un albero di derivazione con radice x_i contenente un simbolo di P ; componendo questo albero come sottoalbero di x_i all'interno di una derivazione di x che applica la produzione considerata, si otterrebbe un albero di derivazione con radice x contenente un simbolo di P , contraddicendo $x \in U$. Dunque tutti i successori di x , per ogni produzione applicabile, appartengono a U , cioè $x \in \bigcirc_G U$.

Essendo U un invariante contenuto in $X \setminus P$, per massimalità di $\Box$ si ha $U \subseteq \Box(X \setminus P) = \neg \Diamond P$ (dove l'ultima uguaglianza segue dalla definizione $\Diamond P = \neg \Box \neg P$). Equivalentemente,

$\diamond P \subseteq X \setminus U$, cioè $\diamond P$ contiene ogni simbolo x tale che $x \notin U$, ovvero ogni simbolo da cui esiste un albero di derivazione contenente un simbolo di P . Questo è precisamente l'enunciato della proposizione. $\square$

La proposizione di raggiungibilità formalizza una domanda classica nell'analisi delle grammatiche: quali simboli sono raggiungibili a partire dal simbolo iniziale S ? Un simbolo z è raggiungibile da S precisamente quando $S \in \diamond\{z\}$. Le grammatiche in cui ogni simbolo di $N \cup \Sigma$ è raggiungibile da S sono dette **ridotte** [3, 1]: ogni simbolo non raggiungibile può essere eliminato dalla grammatica, insieme a tutte le produzioni che lo coinvolgono, senza alterare il linguaggio generato da S . La caratterizzazione tramite $\diamond$ rende esplicito, in termini puramente coalgebrici, l'algoritmo classico di eliminazione dei simboli inutili in una grammatica libera dal contesto.

Esempio 4.2. Si consideri la grammatica $G = (\{S, A, B, C\}, \{a, b\}, P, S)$ con produzioni:

$$S \rightarrow aA \mid b, \quad A \rightarrow aA \mid B, \quad B \rightarrow bB, \quad C \rightarrow a \mid SC.$$

Applicando e_G , le espansioni sono $e_G(S) = \kappa_2(\{aA, b\})$, $e_G(A) = \kappa_2(\{aA, B\})$, $e_G(B) = \kappa_2(\{bB\})$, $e_G(C) = \kappa_2(\{a, SC\})$.

Calcolando $\diamond\{X\}(S)$ per ciascun simbolo X , si trova che S raggiunge A (tramite la produzione $S \rightarrow aA$) e, tramite A , anche B (tramite $A \rightarrow B$); dunque A e B sono raggiungibili da S . Il simbolo C , invece, non compare in nessuna espansione di S , A o B , e dunque $S \notin \diamond\{C\}$: nessun albero di derivazione con radice S può mai contenere C , coerentemente con il fatto che C non appare in nessuna produzione degli altri simboli.

D'altra parte, come si vedrà nella sottosezione seguente, i simboli A e B non sono *generativi*: ogni loro espansione richiede, direttamente o indirettamente, un'ulteriore occorrenza di B , senza mai raggiungere una sequenza di soli simboli terminali. Combinando le due analisi, nessuno dei simboli A, B, C è utile (raggiungibile e generativo): possono essere eliminati insieme a tutte le produzioni che li coinvolgono, riducendo G alla grammatica equivalente con l'unica produzione $S \rightarrow b$, che genera il linguaggio $\{b\}$ — lo stesso linguaggio generato da G , poiché ogni derivazione che sceglie $S \rightarrow aA$ non può mai terminare in una stringa di soli terminali.

4.2.4 Ricorsività

Un'altra proprietà strutturale classica delle grammatiche [3], la presenza di simboli **ricorsivi**, si esprime anch'essa tramite $\diamond$, con un piccolo accorgimento necessario per escludere il caso banale.

Definizione 4.2 (Non terminale ricorsivo). Un simbolo $A \in N$ è **ricorsivo** se esiste una produzione $A \rightarrow \alpha$ e un simbolo x che compare in α tale che $x = A$ oppure $A \in \diamond\{x\}$.

La clausola $x = A$ cattura la ricorsività *immediata* (una produzione di A contiene A stesso nel membro destro), mentre la clausola $A \in \diamond\{x\}$ cattura la ricorsività *indiretta*, tramite

un successore x da cui A è nuovamente raggiungibile. Si noti che non è corretto definire semplicemente “ A è ricorsivo se $A \in \diamond\{A\}$ ”: per come $\diamond$ è stato costruito come duale del massimo invariante $\square$, e poiché ogni invariante Q soddisfa banalmente $A \in Q \Rightarrow A \in \bigcirc_G Q$ senza che ciò richieda alcun passo di derivazione, la relazione $A \in \diamond\{A\}$ rischierebbe di non distinguere un simbolo genuinamente ricorsivo da un artefatto della definizione; richiedere che il primo passo avvenga verso un successore x esplicito, e solo da lì eventualmente si richiuda su A , garantisce che venga catturata una derivazione di lunghezza almeno 1.

Un simbolo ricorsivo utile è precisamente ciò che, combinato con la sua raggiungibilità da S e la sua generatività (introdotta nella sottosezione successiva), garantisce che il linguaggio generato da G sia infinito: la porzione di derivazione da A a se stesso può essere ripetuta un numero arbitrario di volte.

4.3 Macchine di Turing

Il terzo esempio riguarda le macchine di Turing. A differenza degli automi a stati finiti, la cui transizione è totale e deterministica, una macchina di Turing può bloccarsi senza aver terminato (*parzialità* della funzione di transizione), può avere più mosse applicabili in una data configurazione (*non determinismo*), e la sua computazione può non arrestarsi mai. Occorre dunque un funtore che generalizzi $D_A(X) = 2 \times X^A$ sostituendo la componente di transizione totale e deterministica X^A con una componente che ammetta tutte e tre queste possibilità.

4.3.1 Il funtore delle macchine di Turing

Si consideri l’endofuntore su **Set**:

$$T(X) = 2 \times \mathcal{P}(X).$$

Una T -coalgebra $\langle o, \delta \rangle : X \rightarrow 2 \times \mathcal{P}(X)$ consiste, come nel caso di D_A , di una funzione di osservazione $o : X \rightarrow 2$ e di una funzione di transizione $\delta : X \rightarrow \mathcal{P}(X)$, quest’ultima però a valori nell’insieme delle parti anziché in X^A . La differenza rispetto a D_A è duplice: $\delta(x)$ può contenere più di un elemento (non determinismo: più stati successori sono possibili da x) oppure può essere l’insieme vuoto (parzialità: nessuna transizione è applicabile da x , e la computazione si blocca in x). In questo modo $\mathcal{P}(X)$ codifica in un’unica componente sia il non determinismo sia la parzialità, senza bisogno di un funtore separato per ciascuno dei due fenomeni.

Un omomorfismo tra due T -coalgebre $\langle o, \delta \rangle : X \rightarrow 2 \times \mathcal{P}(X)$ e $\langle o', \delta' \rangle : Y \rightarrow 2 \times \mathcal{P}(Y)$ è una funzione $f : X \rightarrow Y$ tale che $o = o' \circ f$ e $f[\delta(x)] = \delta'(f(x))$ per ogni $x \in X$, dove $f[\delta(x)] = \{f(x') \mid x' \in \delta(x)\}$ denota l’immagine diretta di $\delta(x)$ lungo f .

4.3.2 Macchine di Turing come coalgebre

Una macchina di Turing (non deterministica) è specificata da un insieme di stati Q , un alfabeto di nastro Γ con simbolo blank, e una funzione di transizione $\delta_Q : Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$, insieme a un insieme $Q_{\text{acc}} \subseteq Q$ di stati accettanti, dai quali per convenzione non è definita alcuna transizione ulteriore.

Una **configurazione** è una tripla $c = (q, \tau, i)$, dove $q \in Q$ è lo stato corrente, $\tau : \mathbb{N} \rightarrow \Gamma$ è il contenuto del nastro (con supporto finito rispetto al blank) e $i \in \mathbb{N}$ è la posizione della testina. Sia C l'insieme di tutte le configurazioni. La macchina induce canonicamente una T -coalgebra $\langle o_M, \delta_M \rangle : C \rightarrow 2 \times \mathcal{P}(C)$:

$$o_M(q, \tau, i) = \begin{cases} 1 & \text{se } q \in Q_{\text{acc}}, \\ 0 & \text{altrimenti,} \end{cases}$$

$$\delta_M(q, \tau, i) = \{(q', \tau[i \mapsto b], i + d) \mid (q', b, d) \in \delta_Q(q, \tau(i))\}.$$

Si noti che $\delta_M(q, \tau, i) = \emptyset$ esattamente quando $\delta_Q(q, \tau(i)) = \emptyset$, incluso il caso $q \in Q_{\text{acc}}$: la parzialità della funzione di transizione della macchina è così codificata direttamente dalla possibilità che la componente $\mathcal{P}(C)$ sia vuota, esattamente come una parola priva di produzioni applicabili in un simbolo non terminale codifica, nel funtore delle grammatiche, l'assenza di ulteriori espansioni.

Esempio 4.3. Si consideri una macchina che, ricevuto in input un numero n in notazione unaria, verifica se n è composto: in ogni configurazione con stato non deterministico di “ipotesi”, la macchina sceglie non deterministicamente un divisore candidato $2 \leq k < n$ e prosegue verificandolo. La coalgebra corrispondente ha, in tali configurazioni, $\delta_M(c)$ con tanti elementi quanti sono i valori di k ammissibili; se nessun k porta a una configurazione accettante, la macchina non accetta lungo nessuna delle diramazioni.

4.3.3 Accettazione non deterministica tramite $\diamond$

Sia $\text{Acc} = \{c \in C \mid o_M(c) = 1\} \subseteq C$ l'insieme delle configurazioni accettanti. Una **computazione** da c è una sequenza finita $c = c_0, c_1, \dots, c_n$ con $c_{k+1} \in \delta_M(c_k)$ per ogni $k < n$; si dice che M **accetta** da c se esiste una computazione da c con $c_n \in \text{Acc}$.

Proposizione 4.8. M accetta da c se e solo se $c \in \diamond \text{Acc}$.

Dimostrazione. ($\Leftarrow$) Si mostra prima che $\text{Acc} \subseteq \diamond \text{Acc}$: questo segue dalla proprietà generale $P \subseteq \diamond P$, conseguenza immediata di $\Box Q \subseteq Q$ applicata a $Q = \neg P$. Si mostra poi che, se $c' \in \delta_M(c)$ e $c' \in \diamond \text{Acc}$, allora $c \in \diamond \text{Acc}$: infatti $\Box(\neg \text{Acc})$ è esso stesso un invariante (essendo il massimo invariante contenuto in $\neg \text{Acc}$), dunque $c \in \Box(\neg \text{Acc})$ implica $\delta_M(c) \subseteq \Box(\neg \text{Acc})$; per contrapposizione, se un successore c' non è in $\Box(\neg \text{Acc}) = \neg \diamond \text{Acc}$, allora nemmeno c lo è, cioè $c \in \diamond \text{Acc}$. Applicando questi due fatti a ritroso lungo una computazione accettante $c_0, \dots, c_n$ con $c_n \in \text{Acc} \subseteq \diamond \text{Acc}$, si ottiene $c_k \in \diamond \text{Acc}$ per ogni k , in particolare per $k = 0$.

($\Rightarrow$) Sia $U = \{c \in C \mid \text{non esiste alcuna computazione accettante da } c\}$. Si ha $U \subseteq C \setminus \text{Acc}$, poiché una configurazione accettante ammette banalmente la computazione di lunghezza zero. Si mostra che U è un invariante: se $c \in U$, allora per ogni $c' \in \delta_M(c)$ non può esistere una computazione accettante da c' (altrimenti, componendola con il passo $c \rightarrow c'$, si otterrebbe una computazione accettante da c), dunque $\delta_M(c) \subseteq U$, cioè $c \in \bigcirc_M U$. Per massimalità di $\Box$, $U \subseteq \Box(C \setminus \text{Acc}) = \neg \diamond \text{Acc}$, cioè $\diamond \text{Acc} \subseteq C \setminus U$: ogni configurazione in $\diamond \text{Acc}$ ammette una computazione accettante. $\square$

Il linguaggio riconosciuto da M a partire da una configurazione iniziale $c_0(w)$ codificante l'input w è dunque $L(M) = \{w \mid c_0(w) \in \diamond \text{Acc}\}$: la nozione usuale di linguaggio Turing-riconoscibile, in cui basta l'esistenza di *una* diramazione accettante, è precisamente il predicato coalgebrico $\diamond \text{Acc}$.

4.3.4 Bisimulazione ed equivalenza di macchine

Un vantaggio della descrizione coalgebrica è che permette di dimostrare l'equivalenza tra due varianti di macchina di Turing (che differiscono, ad esempio, nella struttura del nastro) esibendo una bisimulazione tra le rispettive coalgebre delle configurazioni, invece di un argomento per induzione sul numero di passi di computazione. Il seguente lemma generale rende preciso il legame tra omomorfismi, bisimulazione e il predicato $\diamond$.

Lemma 4.1. Sia $f : (X, \langle o, \delta \rangle) \rightarrow (Y, \langle o', \delta' \rangle)$ un omomorfismo di T -coalgebre tale che $o = o' \circ f$, e sia $\text{Acc} = o^{-1}(1)$, $\text{Acc}' = o'^{-1}(1)$. Allora, per ogni $x \in X$:

$$x \in \diamond \text{Acc} \iff f(x) \in \diamond \text{Acc}'.$$

Dimostrazione. Il grafico $R = \{(x, f(x)) \mid x \in X\} \subseteq X \times Y$ è una bisimulazione tra $\langle o, \delta \rangle$ e $\langle o', \delta' \rangle$: la struttura di coalgebra su R è data trasportando δ tramite la biiezione $R \cong X$, e le due proiezioni $\pi_1 : R \rightarrow X$ e $\pi_2 : R \rightarrow Y$ sono omomorfismi, la seconda perché $\pi_2 = f \circ \pi_1$ ed f è per ipotesi un omomorfismo.

Si mostra ora $\Rightarrow$ (l'altra direzione è simmetrica, scambiando i ruoli di X e Y tramite la stessa relazione R). Sia $U = \{x \in X \mid f(x) \notin \diamond \text{Acc}'\}$. Si mostra che U è un invariante di $\langle o, \delta \rangle$ contenuto in $X \setminus \text{Acc}$: se $x \in U$, allora $f(x) \in \Box(\neg \text{Acc}')$, che è un invariante, dunque $\delta'(f(x)) \subseteq \Box(\neg \text{Acc}')$; per la condizione di omomorfismo $f[\delta(x)] = \delta'(f(x))$, ogni $x' \in \delta(x)$ soddisfa $f(x') \in \Box(\neg \text{Acc}') \subseteq \neg \diamond \text{Acc}'$, cioè $x' \in U$; dunque $\delta(x) \subseteq U$, cioè $x \in \bigcirc_M U$. Inoltre $U \subseteq X \setminus \text{Acc}$, poiché $o(x) = 1$ implica $o'(f(x)) = 1$, cioè $f(x) \in \text{Acc}' \subseteq \diamond \text{Acc}'$, escludendo $x \in U$. Per massimalità di $\Box$, $U \subseteq \Box(X \setminus \text{Acc}) = \neg \diamond \text{Acc}$: dunque $x \notin \diamond \text{Acc}$ implica $f(x) \notin \diamond \text{Acc}'$, che è la contrapposta della direzione cercata. $\square$

Si applica ora il lemma a un caso concreto: l'equivalenza tra una macchina con nastro bi-infinito e una con nastro mono-infinito, ottenuta tramite la classica costruzione a due tracce.

Proposizione 4.9. Sia M una macchina con nastro bi-infinito, cioè con configurazioni $(q, \tau, i) \in Q \times \Gamma^{\mathbb{Z}} \times \mathbb{Z}$, e sia M' la macchina con nastro mono-infinito ottenuta dalla costruzione a due tracce: lo stato di M' è una coppia $(q, s) \in Q \times \{+, -\}$ che ricorda il segno della posizione corrente, il nastro di M' ha alfabeto $\Gamma \times \Gamma$ e la cella $n \in \mathbb{N}$ contiene la coppia $(\tau(n), \tau(-n-1))$, e la posizione della testina di M' è $|i|$. Allora, per ogni configurazione c di M , $c \in \diamond \text{Acc}$ se e solo se la configurazione corrispondente $e(c)$ di M' soddisfa $e(c) \in \diamond \text{Acc}'$.

Dimostrazione. Si definisce $e(q, \tau, i) = ((q, \text{sgn}(i)), \tau', |i|)$, dove $\tau'(n) = (\tau(n), \tau(-n-1))$ per ogni $n \in \mathbb{N}$, e si estende la funzione di transizione di M' in modo che un passo di M' legga la componente di $\tau'(|i|)$ indicata dal segno corrente, applichi la transizione di M , scriva sulla stessa componente, e aggiorni posizione e segno coerentemente con il nuovo valore di $i \pm 1$ (incluso il cambio di segno quando i attraversa lo zero). Per costruzione, e è un omomorfismo di T -coalgebre: $o_{M'}(e(c)) = o_M(c)$, poiché lo stato accettante di M' dipende solo dalla componente Q , e $\delta_{M'}(e(c)) = e[\delta_M(c)]$, poiché ogni passo di M corrisponde esattamente a un passo di M' sulla traccia appropriata, senza necessità di più passi intermedi. La tesi segue allora immediatamente dal lemma precedente. $\square$

Il punto metodologico è che la correttezza della costruzione a due tracce, tradizionalmente dimostrata verificando per induzione sul numero di passi che le due macchine restano sincronizzate, si riduce alla verifica delle due condizioni di omomorfismo su e , seguita dall'applicazione di un unico lemma generale sulla bisimulazione. Lo stesso schema si applica, con encoding diversi, alla simulazione di macchine multi-nastro, alla riduzione a un alfabeto binario, o a qualunque altra variante della macchina di Turing la cui equivalenza con quella standard sia dimostrata tramite una funzione di codifica delle configurazioni.

Bibliografia

- [1] Alfred V. Aho, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 1986.
- [2] Janusz A. Brzozowski. Derivatives of regular expressions. *Journal of the ACM*, 11(4):481–494, 1964.
- [3] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley, 3rd edition, 2006.
- [4] Bart Jacobs. *Introduction to Coalgebra: Towards Mathematics of States and Observation*. Cambridge Tracts in Theoretical Computer Science. Cambridge University Press, 2017.
- [5] Jan J. M. M. Rutten. Universal coalgebra: A theory of systems. *Theoretical Computer Science*, 249(1):3–80, 2000.
- [6] Michael Sipser. *Introduction to the Theory of Computation*. Cengage Learning, 3rd edition, 2013.